

Trabalho final de Computação gráfica

Lucas de Souza Cerveira Pereira - 22401904

1

1. Introdução

A síntese de imagens realistas em computação gráfica depende, em grande medida, da forma como a interação entre luz e superfície é modelada matematicamente. Ainda que técnicas modernas de iluminação global e sombreamento por fragmento (*per-pixel shading*) sejam hoje o padrão em aplicações de tempo real, o estudo dos modelos clássicos de iluminação local (ambiente, Lambertiano e Blinn-Phong) permanece central na formação em computação gráfica, pois constitui a base conceitual sobre a qual as técnicas contemporâneas foram construídas.

Este trabalho apresenta o desenvolvimento de uma aplicação interativa em OpenGL, utilizando a *Fixed Function Pipeline* (FFP), com o propósito de implementar e comparar visualmente os três modelos de iluminação exigidos pelo enunciado da disciplina: iluminação ambiente, modelo Lambertiano e modelo especular, tendo sido escolhida a variante Blinn-Phong como implementação do componente especular. A aplicação permite ao usuário alternar dinamicamente entre esses modos, reposicionar a fonte de luz, ajustar sua intensidade e trocar os materiais atribuídos aos objetos da cena, observando em tempo real o efeito de cada parâmetro sobre a aparência final da imagem renderizada.

O restante deste documento está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica necessária à compreensão dos modelos implementados; a Seção 3 detalha os modelos de iluminação efetivamente implementados no projeto; a Seção 4 descreve a cena 3D construída; a Seção 5 apresenta a arquitetura e as decisões de implementação; a Seção 6 descreve os experimentos realizados; a Seção 7 apresenta e discute os resultados obtidos; a Seção 8 relata as dificuldades técnicas identificadas; a Seção 9 conclui o trabalho, incluindo limitações e possíveis melhorias futuras; e a Seção 10 apresenta as referências bibliográficas utilizadas.

2. Fundamentação teórica

2.1. Rendering e o pipeline gráfico

Rendering, ou renderização, é o processo pelo qual uma descrição matemática de uma cena tridimensional (composta por geometria, materiais, câmera e fontes de luz) é convertida em uma imagem bidimensional *raster*, isto é, em uma matriz de pixels coloridos [Foley et al. 1990]. Em aplicações de tempo real, esse processo é executado repetidamente, a cada quadro, por uma sequência de estágios conhecida como *pipeline* gráfico, que transforma vértices do espaço do objeto até o espaço de tela e, em seguida, determina a cor final de cada fragmento visível [Akenine-Möller et al. 2018].

No projeto desenvolvido, o rendering é realizado por meio da *Fixed Function Pipeline* do OpenGL clássico, um modelo de pipeline não programável em que os estágios de transformação e iluminação são configurados por meio de estados fixos da API (como *glLightfv* e *glMaterialfv*), em contraste com pipelines modernas baseadas em *shaders* programáveis [Shreiner et al. 2013].

2.2. Modelos de iluminação e a equação de iluminação local

Um modelo de iluminação é uma formulação matemática que estima a quantidade e a cor da luz refletida por um ponto da superfície em direção ao observador, em função das propriedades do material, da orientação da superfície e da posição relativa entre a fonte de luz e o observador [Foley et al. 1990]. Os modelos de iluminação local, empregados neste trabalho, calculam essa contribuição luminosa considerando apenas a interação direta entre a fonte de luz e o ponto observado, sem levar em conta oclusões, sombras projetadas por outros objetos ou múltiplos rebatimentos de luz no ambiente.

A equação de iluminação local clássica, utilizada como base conceitual do projeto, decompõe a cor final de um ponto na soma de três termos:

$$I = k_a L_a + k_d L_d (\mathbf{N} \cdot \mathbf{L}) + k_s L_s (\mathbf{N} \cdot \mathbf{H})^n \quad (1)$$

em que k_a , k_d e k_s representam, respectivamente, os coeficientes de reflexão ambiente, difusa e especular do material; L_a , L_d e L_s são as intensidades luminosas correspondentes emitidas pela fonte; $\mathbf{N}$ é o vetor normal à superfície; $\mathbf{L}$ é o vetor unitário na direção da fonte de luz; $\mathbf{H}$ é o vetor bissetor entre a direção da luz e a direção do observador; e n é o expoente de *shininess*, que controla a concentração do brilho especular. Cada um dos três termos dessa equação corresponde a um dos modos de iluminação implementados no projeto e descritos nas subseções seguintes.

2.3. O papel da normal de superfície

O vetor normal $\mathbf{N}$, perpendicular à superfície no ponto considerado, é o parâmetro geométrico fundamental de qualquer modelo de iluminação local, pois é ele que determina o ângulo de incidência da luz sobre a superfície e, conseqüentemente, a intensidade da reflexão difusa e especular [Angel and Shreiner 2011]. Superfícies cuja normal aponta diretamente para a fonte de luz recebem iluminação máxima, enquanto superfícies cuja normal se afasta progressivamente da direção da luz recebem iluminação decrescente, até o ponto em que a luz deixa de incidir sobre a face.

No projeto, o tratamento da normal varia conforme a geometria: em superfícies curvas, como as esferas, a normal em cada vértice corresponde ao próprio vetor de posição normalizado em relação ao centro, o que produz uma transição suave de iluminação sobre a superfície; em superfícies planas subdivididas, como o cubo e o plano, a normal é constante em cada face, mas a subdivisão da malha em múltiplos vértices permite uma interpolação de cor mais precisa, tema retomado na Seção 5.2.

2.4. Iluminação ambiente

A iluminação ambiente representa uma aproximação simplificada da luz indireta que satura um ambiente fechado, atingindo uniformemente todas as superfícies independentemente de sua orientação ou da posição da fonte de luz. Matematicamente, corresponde ao termo $k_a L_a$ da Equação 1, não dependendo de nenhum vetor direcional. Isoladamente, esse termo produz uma aparência bidimensional e uniforme dos objetos, sem qualquer indício de volume ou profundidade, já que não há variação de intensidade sobre a superfície.

2.5. O modelo Lambertiano (reflexão difusa)

O modelo Lambertiano, proposto originalmente por Johann Heinrich Lambert, descreve superfícies foscas que refletem a luz incidente igualmente em todas as direções, de modo que o brilho percebido depende apenas do ângulo entre a normal da superfície e a direção da fonte de luz, e não da posição do observador [Foley et al. 1990]. Esse comportamento é modelado pelo termo $k_d L_d (\mathbf{N} \cdot \mathbf{L})$ da Equação 1, em que o produto escalar entre os vetores normal e direção da luz (equivalente ao cosseno do ângulo entre eles) determina a intensidade da reflexão difusa. É esse termo que confere aos objetos a percepção de volume, ao criar uma transição gradual entre as regiões mais iluminadas e as regiões em sombra própria.

2.6. O modelo de Phong e o modelo Blinn-Phong

O modelo especular de Phong, introduzido por Bui Tuong Phong em 1975, complementa a reflexão difusa ao simular o brilho concentrado ("highlight") observado em superfícies lisas e polidas [Phong 1975]. Nesse modelo original, o termo especular é calculado a partir do ângulo entre o vetor de reflexão perfeita da luz em relação à normal e o vetor que aponta para o observador, elevado a uma potência n que controla o tamanho e a nitidez do brilho.

O modelo Blinn-Phong, proposto por James Blinn em 1977 como uma reformulação computacionalmente mais eficiente do modelo de Phong, substitui o cálculo do vetor de reflexão perfeita pelo cálculo de um vetor bissetor $\mathbf{H}$ entre a direção da luz e a direção do observador [Blinn 1977]. Esse vetor auxiliar é obtido de forma mais simples, e o termo especular passa a ser calculado como $k_s L_s (\mathbf{N} \cdot \mathbf{H})^n$, conforme apresentado na Equação 1. Por sua eficiência e por ser diretamente suportado pela *Fixed Function Pipeline* do OpenGL, o modelo Blinn-Phong foi o modelo escolhido para a implementação do componente especular deste projeto, atendendo à exigência do enunciado de implementar "o modelo de Phong ou Blinn-Phong".

2.7. Iluminação local e iluminação global

Os modelos discutidos até aqui pertencem à categoria de iluminação local, na qual a cor de um ponto é calculada considerando exclusivamente a contribuição direta das fontes de luz definidas na cena, ignorando efeitos como sombras projetadas por outros objetos, reflexões indiretas entre superfícies e refração [Akenine-Möller et al. 2018]. Já os modelos de iluminação global, como *ray tracing*, *radiosity* e *path tracing*, buscam simular o transporte completo da luz no ambiente, considerando múltiplos rebatimentos entre superfícies, produzindo resultados fisicamente mais realistas, ao custo de um processamento significativamente mais elevado.

O projeto desenvolvido implementa exclusivamente modelos de iluminação local, o que é consistente com o uso da *Fixed Function Pipeline* do OpenGL clássico, historicamente projetada para operar sob essa categoria de modelos em tempo real. Não há, portanto, cálculo de sombras projetadas entre objetos ou de rebatimento de luz indireta entre as superfícies da cena.

3. Modelos de iluminação implementados

Os três modelos de iluminação exigidos pelo enunciado foram implementados na classe `Luz`, localizada em `src/iluminacao/luz.cpp`, que atua como um invólucro

(*wrapper*) sobre a fonte de luz nativa `GL_LIGHT0` do OpenGL. A alternância entre os modos é realizada em tempo de execução, habilitando ou suprimindo dinamicamente os canais de cor enviados à GPU por meio da função `glLightfv`.

3.1. Modo Ambiente

Ativado pela tecla 1, este modo desliga os canais difuso e especular de `GL_LIGHT0`, enviando valores nulos para ambos, de modo que apenas o termo $k_a L_a$ da equação de iluminação permanece ativo. O resultado visual esperado é uma aparência plana e uniforme dos objetos, sem qualquer indício de profundidade.

3.2. Modo Lambertiano (Difuso)

Ativado pela tecla 2, este modo mantém o canal `GL_DIFFUSE` ativo e suprime o canal especular, de modo que a cor de cada ponto passa a depender do produto escalar entre a normal da superfície e a direção da fonte de luz. O resultado visual esperado é a percepção de volume e sombreamento gradual sobre os objetos.

3.3. Modo Blinn-Phong (Especular)

Ativado pela tecla 3, este modo reativa simultaneamente os três canais (ambiente, difuso e especular), permitindo a observação plena do brilho direcional (*highlight*) característico de superfícies polidas. A Fixed Function Pipeline do OpenGL aproxima o modelo especular por meio do vetor bisetor (*half-vector*), caracterizando o modelo Blinn-Phong descrito na Seção 2.6, com a intensidade do brilho controlada pelo parâmetro `GL_SHININESS`, definido por material.

Cabe destacar que, por se tratar de uma pipeline fixa, a avaliação da equação de iluminação é realizada por vértice, e não por fragmento (pixel), de modo que a aplicação implementa um sombreamento de Gouraud, cujas implicações para a qualidade visual do brilho especular são discutidas na Seção 5.2 e na Seção 8.

4. Descrição da cena 3D

A cena construída para o projeto é composta por seis objetos geométricos distintos, distribuídos em quatro categorias, cada uma associada a um material padrão distinto, conforme resume a Tabela 1.

Tabela 1. Objetos que compõem a cena 3D

Categoria	Quantidade	Material padrão	Finalidade didática
Esfera	2	Plástico Vermelho	Normais contínuas / sombreamento suave
Cone	2	Metal Prateado	Normais oscilantes / polo divergente
Cubo	1	Borracha Azul	Faces planas / contraste com superfícies curvas
Plano	1	Cerâmica Branca	Piso de referência / malha fortemente subdividida

As esferas são construídas a partir de uma malha paramétrica de faixas de latitude, desenhadas com `GL_TRIANGLE_STRIP`, em que a normal de cada vértice corresponde ao seu próprio vetor de posição normalizado, o que resulta em uma variação contínua e perfeitamente suave da iluminação sobre a superfície. Os cones apresentam normais calculadas analiticamente em função do ângulo θ ao redor do eixo, sendo sua

base modelada com anéis de `GL_TRIANGLE_STRIP` em vez de um único leque triangular (`GL_TRIANGLE_FAN`), de modo a evitar deficiências de especularidade no centro da base. O cubo e o plano, por sua vez, são construídos a partir de grades subdivididas de `GL_QUADS` (e não por faces únicas de quatro vértices), decisão de projeto discutida em detalhe na Seção 5.2.

A cena conta ainda com um marcador visual (*MarcadorLuz*), representado por uma pequena esfera estática que indica a posição corrente da fonte de luz no espaço, auxiliando o usuário a compreender a relação entre a posição da luz e o resultado observado sobre os objetos.

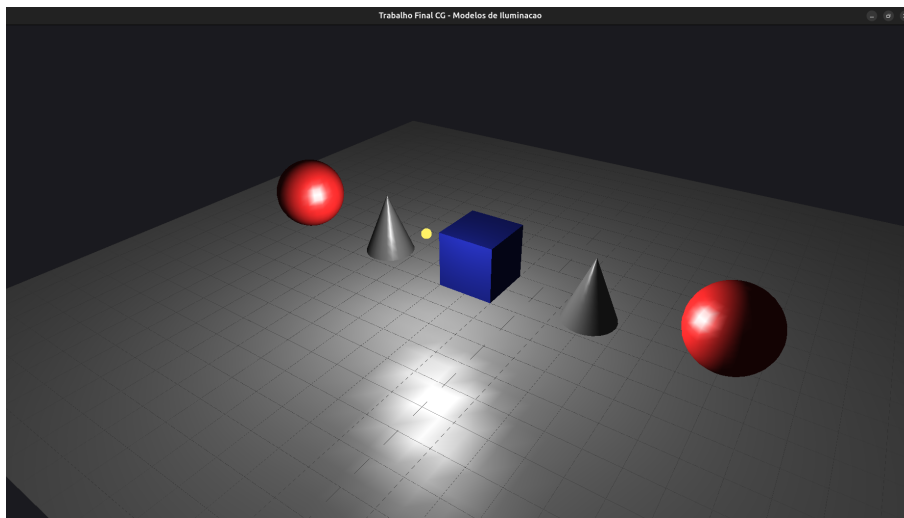


Figura 1. Visão geral da cena 3D com os seis objetos, o piso e o marcador de luz, sob o modelo Blinn-Phong.

5. Descrição da implementação

5.1. Arquitetura geral

O projeto foi implementado em C++ sobre a *Fixed Function Pipeline* do OpenGL, com gerenciamento de janela e eventos por meio da biblioteca FreeGLUT. A organização do código adota princípios de orientação a objetos e separação de responsabilidades, distribuindo as classes em quatro diretórios principais:

- `src/core/`: contém o `main.cpp`, responsável pela inicialização do FreeGLUT, registro de *callbacks* e gerenciamento de estado da interface; a classe *Cena*, que agrega os objetos a serem renderizados; e a classe *Camera*.
- `src/geometria/`: contém a classe base abstrata *Objeto*, que define o contrato polimórfico `desenharGeometria()` e centraliza as propriedades de translação espacial comuns a todas as formas, além das implementações concretas de esfera, cone, cubo e plano.
- `src/iluminacao/`: contém a classe *Luz*, responsável por encapsular o estado da fonte `GL_LIGHT0` e controlar o modo de iluminação ativo.
- `src/materiais/`: contém a classe base *Material*, que agrega os coeficientes de reflexão ambiente, difusa e especular, além do fator de *shininess*; e o *singleton* *GerenciadorMateriais*, responsável por distribuir os materiais aos objetos.

O fluxo de execução da aplicação é impulsionado por um laço assíncrono controlado pelo FreeGLUT: eventos de teclado e mouse alteram variáveis internas de estado e disparam o redesenho da cena por meio de `glutPostRedisplay()`. A cada quadro, a função de desenho limpa os *buffers* de cor e profundidade, reconstrói a matriz da câmera, posiciona a luz e percorre a lista de objetos da Cena, cada um consultando o `GerenciadorMateriais` para obter seu material ativo antes de emitir suas instruções de desenho.

Uma decisão de projeto relevante é a inversão da relação tradicional entre objeto e material: em vez de cada instância de objeto possuir sua própria cor, a cor pertence à categoria do objeto (por exemplo, CUBO), sendo administrada centralmente pelo `GerenciadorMateriais`. Essa abordagem, baseada no padrão de projeto *Singleton* combinado a um dicionário categoria–material, garante que todos os objetos de uma mesma categoria reajam de forma coerente a alterações de material, evitando divergências entre instâncias.

5.2. Sistema de materiais

O sistema de materiais é responsável por fornecer a cada objeto propriedades físicas plausíveis de reflexão, sem exigir o cálculo manual de coeficientes a cada renderização. A classe `Material` armazena quatro parâmetros submetidos à GPU por meio de `glMaterialfv`: os coeficientes ambiente, difuso e especular, cada um representado por quatro componentes RGBA, e o fator de *shininess*, limitado a 128,0.

O `GerenciadorMateriais`, implementado como *singleton* com inicialização preguiçosa (*lazy initialization*), mantém um mapa (`std::map`) que associa cada `CategoriaObjeto` a um `Material` ativo. Foram definidos quatro presets de materiais, resumidos na Tabela 2.

Tabela 2. Presets de materiais implementados

Material	Características
Plástico Vermelho	Reflexão difusa fortemente dominada pela cor do pigmento; reflexão especular acromática (branca); <i>shininess</i> baixa.
Metal Prateado	Reflexão especular tingida (metálica) e componente difusa reduzida; brilho especular de alta intensidade.
Borracha Azul	Superfície fosca; componente especular nula; comportamento inteiramente regido pelo termo Lambertiano.
Cerâmica Branca	Comportamento intermediário; brilho especular esbranquiçado e pouco focalizado; alta capacidade de resposta ao termo ambiente.

A troca de material é realizada pelo usuário por meio da tecla `Tab`, que alterna a categoria de objeto selecionada para edição, e da tecla `4`, que aplica o próximo preset da lista à categoria corrente. As teclas `,` e `.` permitem, respectivamente, reduzir e aumentar o fator de *shininess* da categoria selecionada, possibilitando observar o efeito da concentração do brilho especular em tempo real.

5.3. Sistema de iluminação

O sistema de iluminação, descrito em detalhe na Seção 3, é centralizado na classe `Luz`, que mantém coeficientes difusos e especulares fixos em branco pleno, alternando a cor de emissão para preto conforme o modo de iluminação selecionado pelo usuário suprime cada canal. As trocas de modo são realizadas dinamicamente, enviando *arrays* de valores nulos para os canais indesejados por meio de `glLightfv`. Conforme mencionado, a avaliação da equação de iluminação ocorre por vértice, caracterizando um sombreado de Gouraud.

5.4. Câmera

A classe `Camera` representa uma abstração sobre a função `gluLookAt`, adotando um modelo orbital baseado em coordenadas esféricas (azimute, elevação e raio orbital) convertidas para coordenadas cartesianas por meio de funções trigonométricas a cada quadro. A elevação é limitada ao intervalo de $\pm 89^\circ$, prevenindo o fenômeno de *Gimbal Lock*, que ocorre quando o eixo vertical da câmera se alinha com o eixo de visão, causando comportamentos instáveis de rotação.

O deslocamento lateral da câmera (*Pan*), acionado pelas teclas W, A, S e D, é obtido projetando os vetores "frente" e "direita" no plano XZ em função do ângulo de azimute corrente, de modo que os deslocamentos permaneçam vetorialmente coerentes com a direção para a qual a câmera aponta no momento da interação. Essa escolha evita as instabilidades associadas ao uso de ângulos de Euler fixos por eixo em câmeras de primeira pessoa.

5.5. Interação do usuário

A Tabela 3 resume os controles de interação disponíveis na aplicação.

5.6. Fluxo de renderização

A cada chamada de `glutPostRedisplay()`, a aplicação executa a seguinte sequência: (1) limpeza da cor de fundo por meio de `glClearColor`; (2) limpeza dos *buffers* de cor e profundidade com `glClear`; (3) carregamento da matriz identidade na *ModelView*; (4) reconstrução da matriz *LookAt* a partir do estado corrente da câmera; (5) reposicionamento da fonte de luz e definição de seus parâmetros ativos via `glLightfv`; (6) iteração sobre os objetos da Cena, cada um preservando a matriz corrente com `glPushMatrix()`, aplicando sua translação local, consultando seu material junto ao `GerenciadorMateriais`, executando as instruções geométricas de desenho e restaurando a matriz com `glPopMatrix()`; (7) desenho do marcador visual da luz; e (8) troca dos *buffers* por meio de `glutSwapBuffers()`, exibindo o quadro renderizado.

6. Experimentos realizados

6.1. Experimento 1: Importância do reflexo direcional

Este experimento evidencia a diferença fotométrica entre um material condutor (metal) e um material dielétrico isolante (plástico) sob o modelo especular. O procedimento consistiu em fixar o modo de iluminação em Blinn-Phong (tecla 3) e aplicar, sucessivamente, os

Tabela 3. Controles de interação da aplicação

Controle	Categoria	Ação
Mouse (arrastar, botão esquerdo)	Câmera	Orbita a câmera em torno do ponto de foco
Scroll do mouse / + / -	Câmera	Aplica zoom, alterando o raio orbital
W, A, S, D	Câmera	Deslocamento relativo lateral e frente/trás (Pan)
R	Câmera	Restaura posição e ângulos padrão
1	Iluminação	Ativa o modelo Ambiente
2	Iluminação	Ativa o modelo Lambertiano (Difuso)
3	Iluminação	Ativa o modelo Blinn-Phong (Difuso + Especular)
Setas direcionais	Iluminação	Move a fonte de luz nos eixos x e z
Page Up / Page Down	Iluminação	Move a fonte de luz no eixo y (altura)
[/]	Iluminação	Ajusta a intensidade luminosa global
Tab	Materiais	Alterna a categoria de objeto selecionada
4	Materiais	Aplica o próximo preset de material à categoria selecionada
, / .	Materiais	Reduz/aumenta o fator de <i>shininess</i> da categoria selecionada
ESC	Janela	Encerra a aplicação

materiais Plástico e Metal sobre o objeto da categoria Cubo, mantendo pose e iluminação constantes entre as duas capturas.

As capturas confirmam o comportamento teórico esperado. Com o material Plástico, o cubo exibe uma componente difusa fortemente saturada na cor do pigmento (vermelho), sobre a qual se sobrepõe um *highlight* especular acromático (próximo do branco puro), concentrado na face voltada para a fonte de luz; a face superior, por estar em ângulo desfavorável em relação à luz, aparece quase completamente escura, reforçando a dependência direcional do termo difuso. Já com o material Metal, a componente difusa perde saturação: o corpo do cubo passa a ser dominado por tons de cinza, sem a cor característica de um pigmento, enquanto o *highlight* especular permanece proeminente e de tom acromático/prateado. Essa oposição (cor difusa forte e específica versus cor difusa neutra e apagada, com especular igualmente presente em ambos) evidencia diretamente o uso funcionalmente distinto dos coeficientes k_d e k_s configurados na classe `Material` para cada preset (Seção 5.2). Observa-se ainda, nos dois cones ao fundo (categoria CONE, material Metal Prateado por padrão), que a aparência permanece idêntica entre as duas capturas, confirmando que a alteração de material foi aplicada exclusivamente à categoria Cubo selecionada, evidência prática da coerência garantida pelo `GerenciadorMateriais` centralizado por categoria (Seção 5.1).

6.2. Experimento 2: Avaliação progressiva de Lambert e Blinn-Phong

Este experimento relaciona diretamente a equação de iluminação apresentada na Seção 2.2 com o resultado visual observado, com foco na injeção progressiva de cada termo. O procedimento consistiu em selecionar a Esfera (material padrão Plástico Vermelho) e

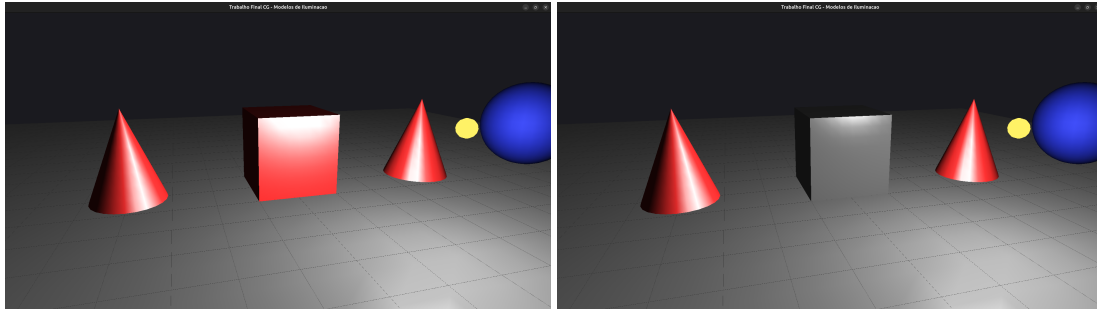


Figura 2. Cubo sob o modelo Blinn-Phong com material Plástico Vermelho (esquerda) e Metal Prateado (direita).

capturar imagens sucessivas ao alternar, em ordem, entre os modos Ambiente (tecla 1), Lambertiano (tecla 2) e Blinn-Phong (tecla 3).

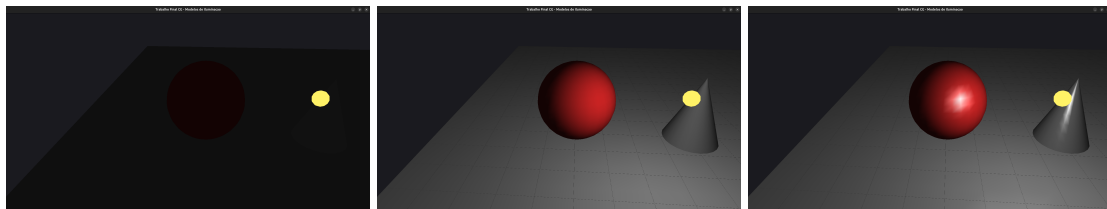


Figura 3. Progressão incremental dos termos de iluminação sobre a Esfera: (a) apenas ambiente; (b) ambiente + difuso; (c) ambiente + difuso + especular.

A sequência confirma o acúmulo progressivo dos termos da equação de iluminação. No modo Ambiente (a), a esfera aparece como um disco vermelho-escuro de intensidade praticamente uniforme, sem qualquer gradiente sobre a superfície, resultado esperado do termo isolado $k_a L_a$, que não depende de nenhum vetor direcional e por isso não revela o volume tridimensional do objeto. No modo Lambertiano (b), surge um gradiente suave e contínuo de claro para escuro, mais intenso na região voltada para a luz e decaindo progressivamente em direção à sombra própria; esse gradiente, resultado do produto escalar $N \cdot L$, é o que confere à esfera sua percepção de volume, favorecido aqui pela variação contínua da normal característica da malha esférica (Seção 2.3). No modo Blinn-Phong (c), soma-se a esse volume já estabelecido um *highlight* pequeno, intenso e bem concentrado, deslocado da região central do gradiente difuso em direção à posição da câmera, evidência direta do termo $(N \cdot H)^n$, cuja concentração está diretamente ligada ao fator de *shininess* do material Plástico.

7. Resultados e discussão

7.1. Visão geral da cena

A Figura 1 (Seção 4), com a câmera recuada e os seis objetos visíveis sob o modelo Blinn-Phong, evidencia a coerência visual entre as diferentes categorias de material: as duas esferas em Plástico Vermelho, os dois cones em Metal Prateado e o piso em Cerâmica Branca exibem, simultaneamente e sob a mesma fonte de luz, respostas fotométricas distintas e consistentes com seus presets, sem qualquer divergência entre instâncias da mesma categoria, resultado direto da centralização do sistema de materiais no GerenciadorMateriais (Seção 5.1).

7.2. Comparação entre materiais

As imagens do Experimento 1 (Figura 2) permitem relacionar diretamente o comportamento observado com os coeficientes de reflexão configurados na classe `Material` (Seção 5.2). O preset `Plástico Vermelho` combina um coeficiente difuso alto e cromático com um coeficiente especular baixo e acromático, produzindo um corpo colorido com brilho discreto e branco. O preset `Metal Prateado` inverte essa relação: coeficiente difuso reduzido e dessaturado, mas resposta especular proeminente, o que explica por que o cubo em Metal perde a cor vermelha de base sem perder o *highlight*. A oposição fotométrica observada nas duas capturas é, portanto, uma consequência direta e verificável da distinção funcional entre k_d e k_s que o projeto se propôs a demonstrar.

7.3. Comparação entre modos de iluminação

As imagens do Experimento 2 (Figura 3) relacionam cada etapa da sequência sobre a Esfera com o termo correspondente da equação de iluminação (Seção 2.2). A transição observada, de um disco opaco, para um volume gradual, para um volume com *highlight* concentrado, confirma empiricamente o comportamento teórico esperado de cada modelo, e a suavidade dessa transição é favorecida pela superfície curva da esfera, cuja normal varia continuamente vértice a vértice.

7.4. Highlights especulares e limitações do sombreamento por vértice

A Figura 4 apresenta uma captura aproximada do brilho especular sobre o Cone com material Metal Prateado.

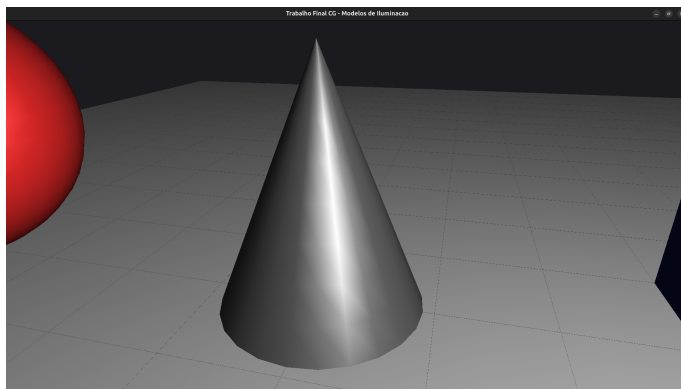


Figura 4. Highlight especular sobre o Cone com material Metal Prateado, evidenciando a faixa vertical de brilho.

Ao contrário de um ponto de brilho circular e isolado (o que se esperaria de um *highlight* especular idealizado), observa-se sobre o Cone uma faixa contínua e alongada de brilho, correndo verticalmente do ápice até a base ao longo da geratriz mais próxima da luz. Esse formato é consequência direta da parametrização analítica da normal do cone (Seção 2.3, Seção 8), que varia apenas em função do ângulo θ ao redor do eixo e permanece constante ao longo de cada geratriz: todos os vértices dispostos verticalmente sobre a geratriz voltada para a luz compartilham, portanto, o mesmo ângulo de incidência, e a interpolação de Gouraud entre esses vértices produz uma faixa de brilho uniforme em vez de um ponto localizado. A subdivisão da base do cone em anéis de `GL_TRIANGLE_STRIP` (em vez de um leque único) garante, adicionalmente, que essa faixa permaneça bem definida e sem degradação de especularidade próximo à base.

7.5. Esfera e Cubo: sombreamento em superfícies curvas versus faces planas subdivididas

A Figura 5 compara o mesmo modelo Blinn-Phong aplicado sobre uma superfície curva (Esfera) e uma superfície de faces planas subdivididas (Cubo).

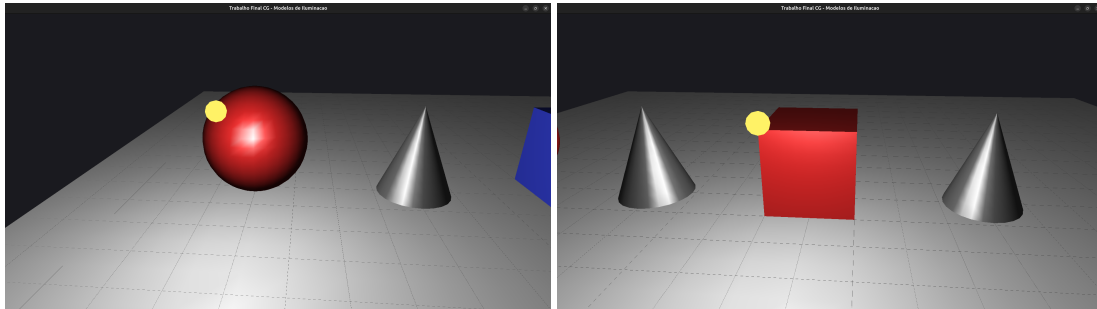


Figura 5. Comparação do sombreamento de Gouraud sobre a Esfera (esquerda) e sobre o Cubo (direita).

Na Esfera, o gradiente de iluminação e o *highlight* especular são perfeitamente contínuos sobre toda a superfície visível, sem qualquer descontinuidade perceptível, consequência da normal variando suavemente entre vértices adjacentes de uma malha densa de faixas de latitude. No Cubo, por outro lado, cada face exibe seu próprio gradiente interno, mas há uma quebra abrupta de intensidade na aresta entre faces adjacentes (visível na transição entre a face superior, quase completamente escura, e a face frontal, fortemente iluminada): como a normal de cada face é constante e as faces são tratadas como grupos de vértices independentes, não há suavização de iluminação entre elas. Essa comparação ilustra concretamente a limitação do sombreamento de Gouraud sobre poucos polígonos grandes discutida na Seção 8, e justifica a decisão de subdividir o Cubo e o Plano em grades densas de `GL_QUADS`: mesmo subdivididas, essas superfícies permanecem estruturalmente planas por face e não reproduzem a continuidade natural obtida na Esfera.

8. Dificuldades encontradas

O desenvolvimento do projeto envolveu três dificuldades técnicas centrais.

A primeira refere-se à contenção do fenômeno de *Gimbal Lock* no sistema de câmera orbital. O cálculo de deslocamento lateral (*Pan*) em um sistema de coordenadas esféricas exige tratamento vetorial cuidadoso, tendo sido necessário limitar o ângulo de elevação da câmera ao intervalo de $\pm 89^\circ$ para evitar comportamentos degenerados de rotação.

A segunda, e mais crítica do ponto de vista fotométrico, diz respeito à acurácia da *Fixed Function Pipeline*. Como a avaliação da equação de iluminação ocorre por vértice (caracterizando o chamado sombreamento de Gouraud) e não por fragmento, a cor final de cada ponto do polígono é obtida por interpolação linear entre os valores calculados em seus vértices, e não por um cálculo independente da equação de iluminação a cada pixel. Essa limitação torna-se particularmente severa em superfícies compostas por poucos polígonos grandes, como seriam um plano ou um cubo representados por apenas quatro vértices por face: nesses casos, um pico de brilho especular pode incidir sobre o interior de

um polígono sem que nenhum de seus vértices esteja em ângulo de colisão adequado com a fonte luminosa, fazendo com que o *highlight* simplesmente desapareça da interpolação, mesmo estando fisicamente presente na superfície. A solução adotada para contornar essa limitação estrutural da pipeline fixa foi subdividir manualmente as classes `Cubo` e `Plano` em grades densas de `GL_QUADS` (com trinta divisões no piso), forçando artificialmente a existência de um número muito maior de vértices interpoladores ao longo de cada face originalmente plana. Essa é uma solução pragmática e não uma correção real do problema: o custo é o aumento artificial do número de vértices processados apenas para compensar uma limitação algorítmica da avaliação por vértice, e mesmo assim o brilho especular resultante nunca é perfeitamente circular ou liso, como seria obtido em um sombreamento por fragmento (*per-pixel lighting*). O tema é retomado na Seção 9.1 como limitação estrutural do projeto.

A terceira dificuldade refere-se à geração analítica da geometria do cone, cuja normal em cada vértice não é obtida automaticamente, exigindo o cálculo manual da função $N(\theta) = \text{normalize}(\text{altura} \cdot \cos \theta, \text{raio}, \text{altura} \cdot \sin \theta)$ para a construção de tiras de triângulos (`GL_TRIANGLE_STRIP`) com orientação anti-horária compatível com as exigências da pipeline fixa.

9. Conclusão

O projeto desenvolvido cumpriu os requisitos estabelecidos pelo enunciado da disciplina, implementando uma aplicação 3D interativa em OpenGL capaz de alternar entre os modelos de iluminação ambiente, Lambertiano e Blinn-Phong sobre uma cena composta por seis objetos de quatro categorias geométricas distintas, cada uma associada a materiais configuráveis em tempo real. A arquitetura adotada, centrada na separação entre geometria, materiais, iluminação e câmera, e na centralização do sistema de materiais em um singleton categorizado, favoreceu a coerência visual e a manutenibilidade do código ao longo do desenvolvimento.

Do ponto de vista didático, a aplicação demonstrou de forma prática a contribuição individual de cada termo da equação de iluminação local (ambiente, difuso e especular), permitindo observar diretamente o efeito de parâmetros como posição e intensidade da luz, material do objeto e fator de *shininess* sobre a aparência final da cena renderizada.

9.1. Limitações atuais

A implementação apresenta limitações inerentes ao uso da *Fixed Function Pipeline*. O sombreamento de Gouraud, por operar sobre valores calculados nos vértices e interpolados linearmente entre eles, produz brilhos especulares que nunca são perfeitamente circulares ou lisos, ao contrário do que se obteria com um sombreamento por fragmento (*per-pixel lighting*). A correção parcial desse problema, por meio da subdivisão excessiva de malhas planas, tem como custo o aumento artificial do número de vértices processados. Adicionalmente, a aplicação depende do gerenciamento estrito de estados da máquina de estados do OpenGL clássico e suporta, no momento, apenas uma única fonte de luz pontual (`GL_LIGHT0`), não havendo suporte simultâneo a múltiplas fontes luminosas.

9.2. Trabalhos futuros

Como melhorias de baixo esforço, destacam-se a adição de uma interface visual sobreposta ao render (por meio de `glutBitmapString`), substituindo os avisos atualmente

impressos apenas no terminal, e a inclusão de um modo de visualização em *wireframe*, acessível via `glPolygonMode`, para avaliação técnica da subdivisão poligonal. Como melhorias de esforço moderado, aponta-se a adição de texturização por mapeamento UV sobre o piso e as esferas, e a refatoração da classe `Luz` para suportar múltiplas fontes de luz simultâneas (`GL_LIGHT1` e adiante). Como melhoria de maior fôlego, destaca-se a migração da *Fixed Function Pipeline* para o *Core Profile* do OpenGL moderno, com implementação de *shaders* programáveis em GLSL, permitindo sombreamento Phong calculado por fragmento e eliminando a necessidade de subdivisão artificial de geometria, além da eventual implementação de *shadow mapping* em tempo real.

Referências

- Akenine-Möller, T., Haines, E., and Hoffman, N. (2018). *Real-Time Rendering*. CRC Press, 4th edition.
- Angel, E. and Shreiner, D. (2011). *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL*. Addison-Wesley, 6th edition.
- Blinn, J. F. (1977). Models of light reflection for computer synthesized pictures. *ACM SIGGRAPH Computer Graphics*, 11(2):192–198.
- Foley, J. D., van Dam, A., Feiner, S. K., and Hughes, J. F. (1990). *Computer Graphics: Principles and Practice*. Addison-Wesley, 2nd edition.
- Phong, B. T. (1975). Illumination for computer generated pictures. *Communications of the ACM*, 18(6):311–317.
- Shreiner, D., Sellers, G., Kessenich, J., and Licea-Kane, B. (2013). *OpenGL Programming Guide: The Official Guide to Learning OpenGL*. Addison-Wesley, 8th edition.